

XCS221 Assignment 1 — Foundations

Due Sunday, August 30 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs221-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset $\text{\LaTeX}$ submission. If you wish to typeset your submission and are new to $\text{\LaTeX}$, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- **hidden:** These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden: Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                           in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

1. Linear Algebra

Linear algebra forms the foundation of modern AI and machine learning. In this problem, you'll work with vectors and matrices using NumPy, which is the standard library for numerical computing in Python and provides efficient implementations of vector and matrix operations that are essential for AI. Understanding these operations is crucial for implementing neural networks, optimization algorithms, and data processing pipelines. For example, the bulk of modern LLMs are just dense matrix multiplications, and NumPy is the first step towards being able to manipulate these matrices. You'll practice basic operations like dot products, matrix multiplication, and distance calculations that appear everywhere in machine learning algorithms.

You'll also learn about Einstein summation notation (einsteinsum), a powerful tool for expressing complex tensor operations concisely.

- (a) [1 point (Written)] **Learn basic NumPy operations with an AI tutor!** Use an AI chatbot (e.g., ChatGPT, Claude, Gemini) to teach yourself basic vector and matrix operations in NumPy (`import numpy as np`). If you'd like more help, [this YouTube video](#) may be helpful. AI tutors have become exceptionally good at creating interactive tutorials, and this year in XCS221, we're testing how they can help you learn fundamentals more interactively than traditional static exercises.

There are many ways to do this. For example, you can simply ask the chatbot "Teach me basic NumPy operations interactively". We provide a prompt template for you to use, but feel free to use your own:

Prompt Template

Teach me basic NumPy operations. Keep the session around 15–20 minutes and interactive.

Guidelines

- Adjust difficulty: if I miss 2 in a row → slow + simpler; if I get 2 quickly → a bit harder or add a twist.
- Use tiny, deterministic examples (small numbers, short tables, brief ASCII diagrams, code blocks). Show actual *run-throughs* (step-by-step states) if helps.
- Use Code if it clarifies: ≤2 Python/NumPy blocks, each ≤20 lines; ideally no imports beyond `import numpy as np`.
- Do NOT solve graded work. If I paste any, refuse and make a similar practice item.
- Be CONCISE. Don't make notation complicated/unnecessary. Focus on intuitions. Code blocks should be short with simple variable names.
- I may interrupt and ask to start somewhere else. Simply adjust and adapt from there.

Suggested flow (flexible; use judgment)

- Start by explaining the topic in a few simple, but technically accurate sentences (explain core idea (≤6 sentences): intuition + minimal notation + why it matters + one common pitfall). Use basic notations and visualizations if necessary (CS221 requires basic math/programming backgrounds). For topics about programming, OK to start with concise code blocks.
- Quick check-in (≤1–2 min): ask 2 short questions about prior exposure and specific goal.
- Exercises: use a tiny instance and show intermediate states. Give exercises that build on each other. OK to give multiple exercises at the same time.
 - e.g., for graph search: a 4–5 node weighted graph; show the frontier/priority-queue after each of 2–3 steps and which edges relax.
 - for DP/MDP/RL/Logic/ML, do an equivalently small, concrete step-by-step trace.
 - If code clarifies, include one concise NumPy block.
- 3 quick checks: increasing difficulty; give a hint first; reveal answers only after I try or ask.
- Assessment and feedback: Give me feedback on my performance - what I understood well, what needs improvement, and specific concepts to review further.
- Session reflection: Ask me for feedback on how the tutoring session went - what worked well, what could be improved, and if the pacing/difficulty was appropriate.

vii. Recap: a 60-second summary

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be around 15–20 minutes and interactive!

(b) [2 points (Written)] **Linear Algebra Complexity.**

Suppose you have two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. What is the time complexity of computing their product AB using the standard matrix multiplication algorithm? Express your answer using big-O notation and briefly justify why. **What we expect:** The time complexity in big-O notation and a 1-2 sentence explanation.

(c) [1 point (Written)] **Learn basic einsum with an AI tutor!**

Use an AI chatbot to teach yourself Einstein summation notation (einsum) in NumPy using the `einops` library (`from einops import einsum, rearrange`). *What is einsum and why?* Einsum notation allows you to express complex array/matrix/tensor operations concisely, including matrix multiplications, tensor contractions, array reshaping, and pretty much everything used in the modern AI models `einops` is a library that allows you to implement einsum notation in Python. Understanding einsum allows you to code more readable code.

Feel free to use the same prompt template from problem 1a, just replace the first sentence with "Teach me how einsum works and how to use the einops library with NumPy". If you'd like more help, [this YouTube video](#) may be helpful.

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be around 15–20 minutes and interactive!

(d) [2 points (Written)] **Einstein Summation**

Given $X \in \mathbb{R}^{n \times d}$ and $\mathbf{w} \in \mathbb{R}^d$: (i) write an `einsum` string for $X\mathbf{w}$; (ii) for the pairwise dot-product matrix $XX^\top$; (iii) for $\text{diag}(X^\top X)$ (column-wise squared norms). Briefly justify each.

What we expect: Provide einsum strings (e.g., `n d, d m -> n m`).

(e) [3 points (Coding)] **Batch Linear Projection (einsum)**

Let $x \in \mathbb{R}^{B \times D_{in}}$ be a batch of input row-vectors, $W \in \mathbb{R}^{D_{in} \times D_{out}}$ a weight matrix, and $b \in \mathbb{R}^{D_{out}}$ a bias vector. We want to compute the following linear transformation in a batched manner: $y[i] = x[i]W + b$ for each batch index i , returning $y \in \mathbb{R}^{B \times D_{out}}$. Use `einsum` (from `einops`) without loops. for the matrix multiplication and NumPy broadcasting for the bias; do not use Python loops.

What we expect: Implement `linear_project(x, W, b)` in `submission.py` with shapes: `x:(B,D_in)`, `W:(D_in,D_out)`, `b:(D_out)` $\rightarrow$ `(B,D_out)`.

(f) [3 points (Coding)] **Split Last Dimension (einops.rearrange pattern string)**

Let $x \in \mathbb{R}^{B \times D}$ and let G divide D evenly. We want to reshape the last axis into G equal chunks, producing $y \in \mathbb{R}^{B \times G \times (D/G)}$. Write the `einops.rearrange` pattern string that performs this reshape. Do not perform the reshape yourself; just return the pattern string. It is fine to assume `D % G == 0`.

What we expect: Implement `split_last_dim_pattern()` in `submission.py` returning a rearrange pattern string (e.g., `'b d -> b g (d/g)'`). The autograder will apply it with the appropriate `g=num_groups`.

(g) [3 points (Coding)] **Normalize Inner Products (einsum)**

Let $A \in \mathbb{R}^{B \times M \times D}$ and $C \in \mathbb{R}^{B \times N \times D}$. For each batch b , we want the matrix $S[b, i, j] = \langle A[b, i, :], C[b, j, :] \rangle$ of all pairwise dot products, giving $S \in \mathbb{R}^{B \times M \times N}$. Use `einsum` (from `einops`) without loops. If `normalize=True`, divide the result by $\sqrt{D}$.

What we expect: Implement `normalized_inner_products(A, C, normalize=True)` in `submission.py` with shapes: `A:(B,M,D)`, `C:(B,N,D)` $\rightarrow$ `(B,M,N)`.

(h) [3 points (Coding)] **Mask Strictly Upper Triangle**

Let `scores` $\in \mathbb{R}^{B \times L \times L}$. For each batch, set entries with column index strictly greater than the row index (i.e., the strictly upper-triangular part where $j > i$) to `-np.inf`, leaving other entries unchanged. Construct the mask using NumPy broadcasting; do not use loops.

For example, if $L = 3$, transform:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & -\infty & -\infty \\ 4 & 5 & -\infty \\ 7 & 8 & 9 \end{pmatrix}$$

What we expect: Implement `mask_strictly_upper(scores)` in `submission.py` for `scores:(B,L,L)`, returning a masked array of the same shape.

(i) **[3 points (Coding)] Probability-Weighted Sum (einops.einsum pattern string)**

Let $P \in \mathbb{R}^{B \times N}$ be batch-wise probability weights (each row sums to 1) and $V \in \mathbb{R}^{B \times N \times D}$ the corresponding value vectors. Provide only the `numpy.einsum` string that computes the weighted sums $out[b, :] = \sum_{j=1}^N P[b, j] V[b, j, :]$, yielding $out \in \mathbb{R}^{B \times D}$.

What we expect: Implement `prob_weighted_sum_einsum()` in `submission.py` that returns the einsum string; the autograder will apply it to `P` and `V`.

2. Calculus and Gradients

Gradients are essential for training machine learning models through optimization algorithms like gradient descent. In this problem, you'll practice computing gradients analytically and verify your results using numerical methods (finite differences).

(a) [2 points (Written)] **Gradient Warmup**

For $f(\mathbf{w}) = \sum_{i=1}^d (w_i - c_i)^2$, derive $\nabla f(\mathbf{w})$. Then evaluate the gradient at $\mathbf{w} = \mathbf{0}$.

What we expect: A compact vector expression (e.g., $(\mathbf{w} + \mathbf{c})$) and one evaluated vector.

(b) [3 points (Coding)] **Gradient Warmup Implementation** Given a vector $\mathbf{w}$ and constants $\mathbf{c}$, compute the gradient $\nabla f(\mathbf{w})$ where $f(\mathbf{w}) = \sum_{i=1}^d (w_i - c_i)^2$.

What we expect: Implement `gradient_warmup(w, c)` in `submission.py` that takes vectors $\mathbf{w}$ and $\mathbf{c}$ and returns the gradient vector.

(c) [3 points (Written)] **Matrix Multiplication Gradient**

Consider two matrices A (size $m \times n$) and B (size $n \times p$) that are multiplied together to form $C = AB$, and then all entries of C are summed to produce a scalar $s = \sum_{i,j} C_{i,j}$.

Let $A = \begin{pmatrix} 2 & 1 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ and $B = \begin{pmatrix} 7 & 8 \\ 9 & 0 \\ 1 & 2 \end{pmatrix}$.

Compute $C = AB$ and $s = \sum_{i,j} C_{i,j}$. Then find the gradient $\frac{\partial s}{\partial A_{i,k}}$ for each entry of matrix A , and similarly find $\frac{\partial s}{\partial B_{k,j}}$ for each entry of matrix B .

What we expect: The computed matrices C and scalar s , plus the gradient matrices $\frac{\partial s}{\partial A}$ and $\frac{\partial s}{\partial B}$ with numerical values for each entry.

(d) [3 points (Coding)] **Matrix Gradient Implementation**

Implement a function that computes the gradients $\frac{\partial s}{\partial A}$ and $\frac{\partial s}{\partial B}$ for the scalar $s = \sum_{i,j} (AB)_{i,j}$ using NumPy operations.

Hint: Feel free to use the `np.ones` or `np.repeat` functions to create the gradient matrices.

What we expect: Implement `matrix_grad(A, B)` in `submission.py` that returns a tuple `(grad_A, grad_B)` where each gradient matrix has the same shape as the corresponding input matrix.

(e) [3 points (Coding)] **Finite Differences**

To build intuition for what gradients really are, we will implement [finite differences](#) to numerically approximate gradients. This approach helps you understand the fundamental definition of derivatives and provides a valuable debugging tool for verifying analytical gradient computations.

The key idea is to approximate the derivative by measuring how much the function changes when we make a small perturbation along each coordinate. The central difference formula is:

$$\frac{\partial f}{\partial w_i} \approx \frac{f(\mathbf{w} + \epsilon \hat{\mathbf{u}}_i) - f(\mathbf{w} - \epsilon \hat{\mathbf{u}}_i)}{2\epsilon}$$

where $\hat{\mathbf{u}}_i$ is a unit vector with 1 in the i -th position and 0 elsewhere, and ϵ is a small step size (like 10^{-5}).

Think of it like checking your speedometer: if you drive a tiny distance and time how long it takes, you can estimate your speed. Similarly, if you move a tiny amount along each coordinate axis and see how the function changes, you can estimate each component of the gradient.

Now, implement two functions to compute the gradient of the least-squares objective $f(\mathbf{w}) = \frac{1}{2} \|\mathbf{A}\mathbf{w} - \mathbf{b}\|_2^2$: one using the analytical formula and one using finite differences for verification. Use NumPy (optionally `einsum`) for vectorized implementation.

Hint: You can use the `np.eye(d)` function to create the unit vectors.

What we expect: In `submission.py`, implement `lsq_grad(w, A, b)` (analytic gradient) and `lsq_finite_diff_grad(w, A, b, epsilon=1e-5)` (central-difference gradient). The autograder will compare them on random instances.

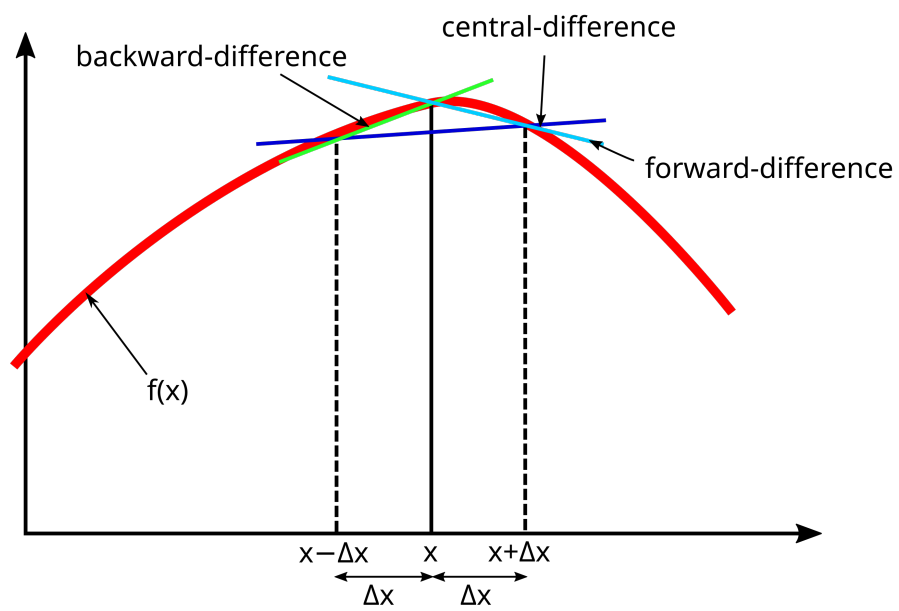


Figure 1: Image from Wikipedia

3. Optimization

Optimization is central to AI - we cast many AI problems as finding the best solution in a rigorous mathematical sense. In this problem, you'll work with analytical optimization techniques and implement them using NumPy to verify your mathematical solutions computationally.

The programming components will help you understand how theoretical optimization translates to practical implementations. You'll implement weighted least squares optimization, explore operator precedence in optimization problems, and use gradient descent to solve quadratic optimization problems numerically.

(a) **[3 points (Written)] Optimization Warmup**

Let $x_1, \dots, x_n$ be real numbers and $w_1, \dots, w_n$ positive weights. Consider the quadratic function: $f(\theta) = \sum_{i=1}^n w_i(\theta - x_i)^2$. Note that θ is a scalar. What value of θ minimizes $f(\theta)$?

Show that the optimum you find is indeed a minimum. What problematic issues could arise if some of the w_i 's are negative?

Note: You can think about this problem as trying to find the point θ that's not too far away from the x_i 's. Over time, hopefully you'll appreciate how nice quadratic functions are to minimize.

What we expect: An expression for the value of θ that minimizes $f(\theta)$ and how you got it. A short calculation/argument to show that it is a minimum. 1-2 sentences describing a problem that could arise if some of the w_i 's are negative.

(b) **[1 point (Written)] Learn about gradient descent with an AI tutor!**

Use an AI chatbot to teach yourself gradient descent optimization techniques.

Gradient descent is a fundamental optimization algorithm used throughout machine learning and AI. It's the backbone of training neural networks and solving many optimization problems. The key idea is that we can find the minimum of a function by repeatedly taking steps in the direction of the negative gradient (steepest descent)—like a ball rolling downhill. Ask your AI tutor to explain the intuition behind why this works, how to choose step sizes, and what can go wrong (like overshooting or getting stuck in local minima).

Feel free to use the prompt template from question 1 with the prompt: "Teach me how gradient descent works on simple and complicated functions."

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be about 15-20 minutes and interactive!

(c) **[3 points (Coding)] Gradient Descent for a 1D Quadratic**

Implement `gradient_descent_quadratic(x, w, theta0, lr, num_steps)` in `submission.py` to minimize the scalar objective $f(\theta) = \sum_{i=1}^n w_i(\theta - x_i)^2$, where $\theta \in \mathbb{R}$ (recall problem 3a). The function should return the final scalar iterate after `num_steps` gradient steps.

What we expect: Implementation of gradient descent to minimize the quadratic function from problem 3a. Your function should perform `num_steps` iterations of gradient descent starting from `theta0` with learning rate `lr`.

4. Ethical Issue Spotting

One of the goals of this course is to teach you how to tackle real-world problems with tools from AI. But real-world problems have real-world consequences. Along with technical skills, an important skill every practitioner of AI needs to develop is an awareness of the ethical issues associated with AI. The purpose of this exercise is to practice spotting potential ethical concerns in applications of AI - even seemingly innocuous ones.

In this question, you will explore the ethics of four different real-world scenarios using the ethics guidelines produced by a machine learning research venue, the NeurIPS conference. The [NeurIPS Ethical Guidelines](#) list seventeen non-exhaustive concerns under General Ethical Conduct and Potential Negative Social Impacts (the numbered lists). For each scenario, you will write a potential negative impacts statement. To do so, you will first determine if the algorithm / dataset / technique could have a potential negative social impact or violate general ethical conduct (again, the seventeen numbered items taken from the [NeurIPS Ethical Guidelines](#) page). If the scenario does violate ethical conduct or has potential negative social impacts, list one concern it violates and justify why you think that concern applies to the scenario. If you do **not** think the scenario has an ethical concern, explain how you came to that decision. Unlike earlier problems in the homework there are many possible good answers. If you can justify your answer, then you should feel confident that you have answered the question well.

Each of the scenarios is drawn from a real AI research paper; you should think about why the researchers may have chosen for the algorithms to behave in the way as described in the scenario. The ethics of AI research closely mirror the potential real-world consequences of deploying AI, and the lessons you'll draw from this exercise will certainly be applicable to deploying AI at scale. As a note, you are not required to read the original papers, but we have linked to them in case they might be useful. Furthermore, you are welcome to respond to anything in the linked article that's not mentioned in the written scenario, but the scenarios as described here should provide enough detail to find at least one concern.

Write a 2-5 sentence paragraph for each of the scenarios where you either A. identify at least one ethical concern from the [NeurIPS Ethical Guidelines](#) and justify why you think it applies, or B. state that you don't think a concern exists and justify why that's the case. Chosen scenarios may have anywhere from zero to multiple concerns that match, but you are only required to pick one concern (if it exists) and justify your decision accordingly. Furthermore, copy out and underline the ethical checklist item to which you are referring as part of your answer (i.e.: Severely damage the environment). We have also included a citation in the example solution below, but you are not required to add citations to your response.

Example Scenario: You work for a U.S. hospital that has recently implemented a new intervention program that enrolls at-risk patients in programs to help address their chronic medical issues proactively before the patients end up in the hospital. The intervention program automatically identifies at-risk patients by predicting patients' risk scores, which are measured in terms of healthcare costs. However, you notice that for a given risk score tier, the Black patients are considerably sicker when enrolled than white patients, even though their assigned illness risk score is identical. You manually re-assign patients' risk scores based on their current symptoms and notice that the percentage of Black patients who would be enrolled has increased from 17% to over 45%. [1]

Example Solution: This algorithm has likely encoded, contains, or potentially exacerbates bias against people of a certain race or ethnicity since the algorithm predicts healthcare costs. Because access to medical care in the U.S. is unequal, Black patients tend to have lower healthcare costs than their white counterparts [2]. Thus the algorithm will incorrectly predict that they are at lower risk.

(a) [1 point (Written)]

An investment firm develops a simple machine learning model to predict whether an individual is likely to default on a loan from a variety of factors, including location, age, credit score, and public record. After looking through their results, you find that the model predicts mainly based on location and that the model mainly accepts loans from urban centers and denies loans from rural applicants [3]. Furthermore, looking at the gender and ethnicity of the applicants, you find that the model has a significantly higher false positive rate for Black and male applicants than for other groups. In a false positive prediction, a model misclassifies someone who does not default as likely to default.

(b) [1 point (Written)]

Stylometry is a way of predicting the author of contested or anonymous text by analyzing the writing patterns in the anonymous text and other texts written by the potential authors. Recently, highly accurate machine

learning algorithms have been developed for this task. While these models are typically used to analyze historical documents and literature, they could be used for deanonymizing a wide range of texts, including code [4].

(c) **[1 point (Written)]**

A research group scraped millions of faces of celebrities off of Google images to develop facial recognition technology [5]. The celebrities did not give permission for their images to be used in the dataset and many of the images are copyrighted. For copyrighted photos, the dataset provides URL links to the original image along with bounding boxes for the face.

(d) **[1 point (Written)]**

Researchers have recently created a machine learning model that can predict plant species automatically directly from a single photo [6]. The model was trained using photos uploaded to the iNaturalist app by users who consented to use of their photos for research purposes, and the model is only used within the app to help users identify plants they might come across in the wild.

5. Product Ethics Exploration: Basic Information

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it. Please note that the questions regarding your chosen product are interconnected and will progressively build upon each other from one assignment to the next.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product’s official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will develop a foundational understanding of your chosen product by exploring its purpose, recent developments, mission alignment, and pricing structure.

Why it’s important: Understanding these basics provides the context needed to analyze deeper issues like safety, fairness, and societal impact in later parts.

Suggested resources:

- Official product documentation and feature pages
- Investor or official product blog posts announcing updates
- News coverage from the last 1–2 months
- Company mission statement (“About Us” page, etc.)
- Product pricing and feature comparison pages

(a) [1 point (Written)] Product Purpose and Usage

- Use your chosen product for at least 20 minutes.
- Include at least two **screenshots** showing your interactions with the product. These should highlight its key purposes, typical task flows, and the core problems it addresses for users. Paste your screenshots into your **PDF writeup for this part**.
- Based on your hands-on use and official resources, describe what the product is designed to do in 4–6 sentences. Identify at least two use cases and provide concrete examples (with screenshots in your PDF) where appropriate.

What we expect: Submit a brief description (4–6 sentences) of the product’s purpose and at least two concrete example user tasks. Include relevant screenshots in your PDF writeup.

(b) [1 point (Written)] Recent News and Developments

- Find and read 3+ recent news articles, official announcements, or blog updates about the product and/or its parent company. These articles need to be published within the last 1–2 months. Summarize your findings in 4–6 sentences, focusing on major new features, company developments, or relevant controversies.

What we expect: Include a 3–5 sentence well-sourced summary (with links/citations) of the latest news and notable developments regarding this product or its parent company.

(c) [1 point (Written)] Company & Product Mission Alignment

- What is the company’s stated mission (quote from their website/official pages)?
- What is the specific goal or intended impact of this particular product?
- How does this product fit (or not fit) into the company’s stated mission?
- Consider the concept of **alignment**: how to make AI do what we want. Does the product behave consistently with its stated goals, or have you observed instances where it might “reward hack” (optimize for

the wrong objective) or produce outputs misaligned with user intentions? How does the company address the challenge of ensuring the AI system’s behavior matches its intended purpose?

What we expect: Briefly state the company’s official mission and the product’s stated goal. Analyze, in your own words (2-3 sentences), the relationship between the two—if you notice any tension, mismatch, or “mission drift,” flag and explain it. Additionally, reflect on alignment challenges: whether the product consistently behaves as intended, and how the company addresses potential misalignment issues.

(d) [1 point (Written)] **Product Pricing and Access**

- What are the pricing details for this product (list the main plans/tiers and their costs)?
- Does the pricing follow an equitable or tiered-access model (e.g., discounts for college students)?
- Beyond financial cost, are there **non-financial barriers** to accessing or using the product (such as device requirements, region restrictions, or mandatory integrations)?
- Suggest at least one alternative pricing model. Reflect on potential pros and cons for both the company and its user base.
- Consider the **ecosystem view** of AI: AI’s impact depends on upstream factors (data, compute, labor, environment) and downstream factors (users, deployment, harm). What can you determine about the computational resources required to run this product? Does the company disclose information about the **environmental impact** of training or operating the system (e.g., energy consumption, carbon footprint)? How might environmental costs relate to pricing and access?

What we expect: In 3-4 sentences, list and describe the core pricing model, note any access barriers (financial or otherwise), and discuss at least one alternative pricing approach with a brief pros/cons analysis. Additionally, discuss what you can determine about the computational and environmental costs of the product, and how these relate to pricing and access.

References

- [1] Ziad Obermeyer, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464):447–453, 2019.
- [2] Institute of Medicine of the National Academies. *Unequal Treatment: Confronting Racial and Ethnic Disparities in Health Care*. National Academies Press, 2003.
- [3] Imperial College London. Loan default prediction dataset. Kaggle Competition, 2014.
- [4] Aylin Caliskan-Islam, Richard Harang, Andrew Liu, Arvind Narayanan, Clare Voss, Fabian Yamaguchi, and Rachel Greenstadt. De-anonymizing programmers via code stylometry. In *24th USENIX Security Symposium*, 2015.
- [5] Omkar M. Parkhi, Andrea Vedaldi, and Andrew Zisserman. Deep face recognition. In *British Machine Vision Conference (BMVC)*, 2015.
- [6] iNaturalist. A new vision model. iNaturalist Blog, 2020.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the —README.md— for this assignment includes instructions to regenerate this handout with your typeset $\text{\LaTeX}$ solutions.

- 1.a
- 1.b
- 1.c
- 1.d

2.a
2.c

3.a

3.b

4.a

4.b

4.c

4.d

5.a

5.b

5.c

5.d